

Basic SSRF Against Another Back-End System

PortSwigger Web Security Academy write-up

Name: Matyas Abraham

ID: CTC-207-26

Lab URL: portswigger.net/web-security/ssrf/lab-basic-ssrf-against-backend-system

Reconnaissance / Enumeration

The shop's product page has a "Check stock" feature, and intercepting that request in Burp showed it isn't just checking stock locally - it POSTs to /product/stock with a parameter called stockApi that holds a full URL:

```
stockApi=http://192.168.0.49:8080/admin
```

That's a big red flag: the front-end server is making its own server-side request to whatever URL is sitting in that parameter. Since the app server can obviously reach its own internal network, that's a direct line into hosts that should never be reachable from the outside - classic SSRF setup. I sent the request to Repeater/Intruder to start probing what else lives on that internal network.

The screenshot displays the Burp Suite interface. On the left, a list of intercepted requests is shown, with the selected request being a POST to /product/stock. The request details are visible, including headers like Host, Cookie, User-Agent, and Referer. The body of the request contains the parameter stockApi=http://192.168.0.49:8080/admin. On the right, the 'Payloads' tab is active, showing configuration options for a simple list of strings. The 'Payload position' is set to 'No payload positions configured', and the 'Payload type' is 'Simple list'. The 'Payload count' and 'Request count' are both 0. The 'Payload configuration' section includes buttons for 'Paste', 'Load...', 'Remove', 'Clear', and 'Deduplicate', along with an 'Add' button and a text input field. The 'Payload processing' section has an 'Add' button and a table with columns for 'Enabled' and 'Rule'. The 'Payload encoding' section has a checkbox for 'URL-encode these characters' and a text input field.

Vulnerability Discovered

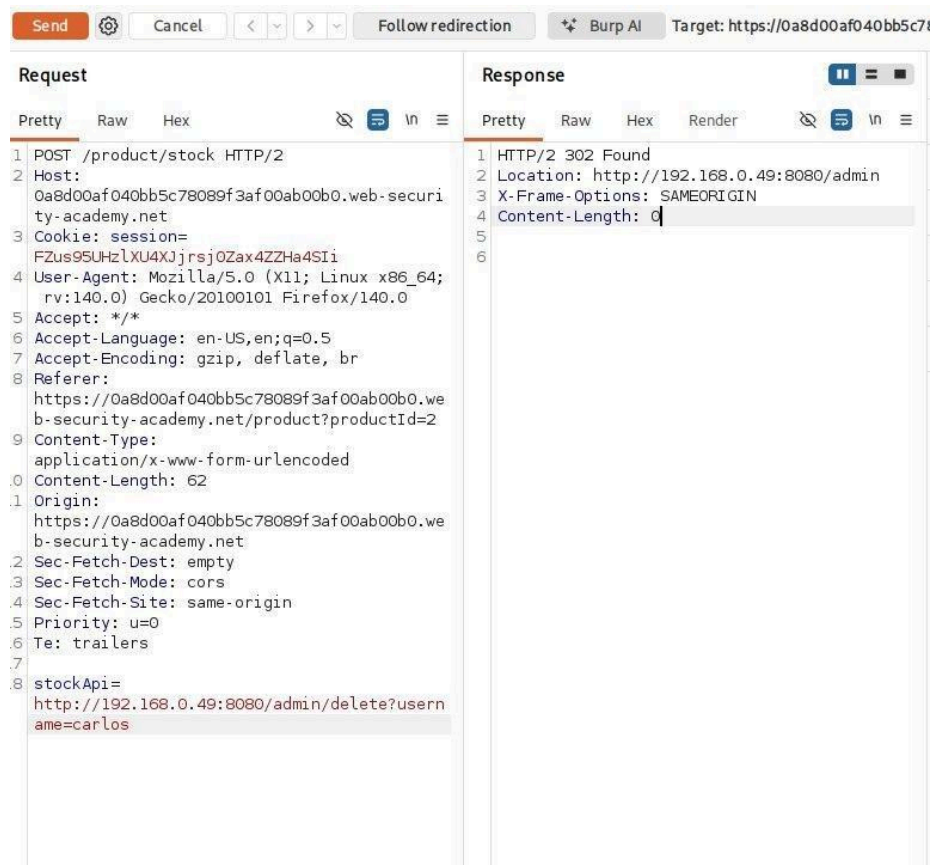
Following the internal IP that stockApi was already pointed at (192.168.0.49:8080), I found an internal admin interface at /admin - the same kind of back-end admin panel that's normally only reachable from inside the network, never from the public internet. This is a Server-Side Request Forgery vulnerability

(CWE-918): the app blindly fetches whatever URL a client supplies in stockApi, so an attacker can make the trusted app server act as a proxy into infrastructure that's otherwise completely walled off from the outside.

Exposed Information & Security Impact

Once the internal admin path was confirmed, I changed the stockApi value to point at the admin panel's delete endpoint instead of just the landing page:

stockApi=http://192.168.0.49:8080/admin/delete?username=carlos



The server actually followed through on it - the response came back as a 302 redirect to the admin panel's homepage, meaning the delete action was carried out server-side without ever needing direct access to that internal panel myself. The impact here is severe: SSRF turned a public-facing product page into a pivot point that let me trigger real admin actions (deleting a user account) on a back-end system that has zero exposure to the internet on its own. Any internal service reachable from the app server - admin panels, cloud metadata endpoints, internal APIs, databases - is fair game once a request parameter like this is trusted blindly.

Remediation Recommendations

1. Never let a client-supplied value control the destination of a server-side request. If the app needs to call a stock API, that URL/host should be hard-coded or pulled from a trusted server-side allowlist - not read from user input at all.
2. Segment internal-only services (like admin panels) onto a network the application/web servers can't reach in the first place, and enforce that separation with firewall rules - not just the assumption that "nobody will find the internal IP."
3. Require authentication on internal admin interfaces regardless of where the traffic appears to originate from. Treating anything coming from inside the network as automatically trusted is exactly what let one SSRF request turn straight into an unauthenticated account deletion here.

Proof of Solve

Screenshot below shows the lab marked as Solved:

